

TecmiPickup

Documentación del proyecto

Aplicación de pedidos de comida para campus con modalidad de recogida en mostrador

Frontend TypeScript · Next.js · Tailwind CSS	Backend Java · Spring Boot · REST · JWT
Hosting frontend Cloudflare	Servidor / API Fly.io

Versión de documentación: 3.0 · 29 de agosto de 2026

1. Descripción general

TecmiPickup es una aplicación de pedidos de comida dentro del campus. El usuario selecciona un local, consulta su menú, configura el pedido, paga y posteriormente recoge el pedido en el mostrador. No existe servicio de delivery.

El sistema busca reducir la fila física y la incertidumbre del usuario: la selección y el pago pueden realizarse desde el teléfono, mientras que el restaurante recibe una cola digital y actualiza el estado de preparación.

El cliente web ya recorre el flujo principal utilizando datos locales. La API se desarrolla en Java con Spring Boot, REST y JWT. El servidor es responsable de la autenticación, catálogo, precios, totales, ETA y órdenes.

Regla de negocio central: el cliente no debe enviar ni imponer precios finales. El servidor recibe la selección del usuario, valida el menú vigente y recalcula el importe antes de aceptar la orden.

2. Problema que resuelve

En el contexto de un campus, ordenar comida suele implicar desplazarse al local, hacer fila, decidir el pedido, pagar y esperar sin conocer con precisión el momento de recogida. TecmiPickup concentra la interacción previa en el teléfono.

Problema	Respuesta de TecmiPickup
Fila para ordenar	El pedido se crea desde el teléfono.
Incertidumbre de preparación	El cliente consulta estado y ETA.
Configuración manual de extras	Opciones estructuradas por producto.
Cola física difícil de consultar	Cola digital para el restaurante.
Validación de entrega	QR o ticket para recoger.

3. Actores y roles

Cliente. Estudiante o visitante con matrícula. Consulta locales y menús, busca productos, marca favoritos, configura pedidos, paga y consulta el estado de sus órdenes.

Restaurante. Consulta la cola del local, inicia la preparación, cambia el estado del pedido y confirma la entrega.

Servidor. Constituye la autoridad del sistema: autentica, autoriza, valida el menú, calcula precios y ETA y mantiene el ciclo de vida de la orden.

Rol	Funciones
Cliente	Catálogo, búsqueda, favoritos, carrito, opciones, pago, seguimiento y pickup.
Restaurante	Cola, preparación, estados y confirmación de entrega.
Servidor/API	Reglas de negocio, autenticación, autorización, validación, precios, ETA y órdenes.

4. Explicación del diseño

La interfaz se diseñó alrededor del comportamiento real de un usuario de campus: decidir rápidamente qué comer, minimizar pasos antes de pagar y mantener visible el estado del pedido. Cada estructura visual responde a una función concreta del flujo.

Estructura	Qué representa	Por qué se eligió
Encabezado / wordmark	Identidad de TecmiPickup y punto de retorno al inicio.	Mantiene una navegación sencilla y una referencia visual constante.
Barra de búsqueda	Entrada rápida para localizar productos o conceptos como café.	Reduce el tiempo necesario para encontrar un platillo sin recorrer todo el catálogo.
Rail de categorías	Filtros como Tacos, Café, Saludable, Pizza, Sushi, Burgers, Postres y Bebidas.	Permite explorar por intención alimentaria y reduce el espacio de búsqueda.
Anuncios/promociones	Destacan promociones y productos de interés.	Aprovechan el inicio para mostrar ofertas sin interferir con el flujo principal.
Tarjetas de local/producto	Muestran nombre, valoración, etiquetas, características y tiempo estimado.	Permiten comparar opciones rápidamente antes de entrar al detalle.
Favoritos	Control de corazón asociado a las tarjetas.	Permite conservar opciones frecuentes.
ETA / tiempo	Indicadores como 5-12 min o 10-20 min.	Ataca directamente la incertidumbre que motiva el proyecto.
Dock inferior	Acceso persistente a Inicio, Buscar, Carrito y Perfil.	Reduce la navegación entre las superficies más utilizadas, especialmente en móvil.
Island de pedido	Superficie persistente para mostrar Recibido → En cocina → Listo.	Mantiene el pedido activo visible sin abandonar la pantalla actual.
QR / ticket	Comprobante asociado al pedido para recoger.	Proporciona una referencia concreta para validar la entrega en mostrador.

Decisión de arquitectura visual: el diseño prioriza contenido y acciones sobre decoración. Las tarjetas, filtros, búsqueda, carrito, estado y pickup forman una cadena continua: descubrir → elegir → configurar → pagar → esperar → recoger.

5. Arquitectura y tecnologías

Capa	Tecnología / responsabilidad
Frontend	TypeScript + Next.js + Tailwind CSS. Interfaz, navegación, catálogo, carrito y estados visuales.
Backend	Java + Spring Boot. API REST y reglas de negocio.
Autenticación	JWT para identificar al usuario y proteger operaciones según rol.
Hosting frontend	Cloudflare para el despliegue del cliente web.
Servidor / API	Fly.io para alojar el backend y exponer los servicios de la API.
Datos	El servidor mantiene la autoridad sobre usuarios, catálogo, pedidos, pagos y estados; el frontend puede utilizar datos locales como fallback para catálogo.

6. Flujo principal

Autenticación → Inicio/Catálogo → Búsqueda o categoría → Ficha del platillo → Configuración → Carrito → Pago → Recibido → En cocina → Listo → QR/ticket → Recogida confirmada.

Estado	Responsable	Descripción
recibido	Sistema/restaurante	Pedido aceptado y colocado en la cola.
en cocina	Restaurante	La preparación está en proceso.
listo	Restaurante	El pedido está disponible para recoger.
entregado	Restaurante	La recogida se confirmó mediante el mecanismo definido.

7. Capturas de pantalla de la ejecución

Las siguientes capturas documentan la ejecución actual del cliente web. El frontend presenta el flujo visual utilizando datos locales.

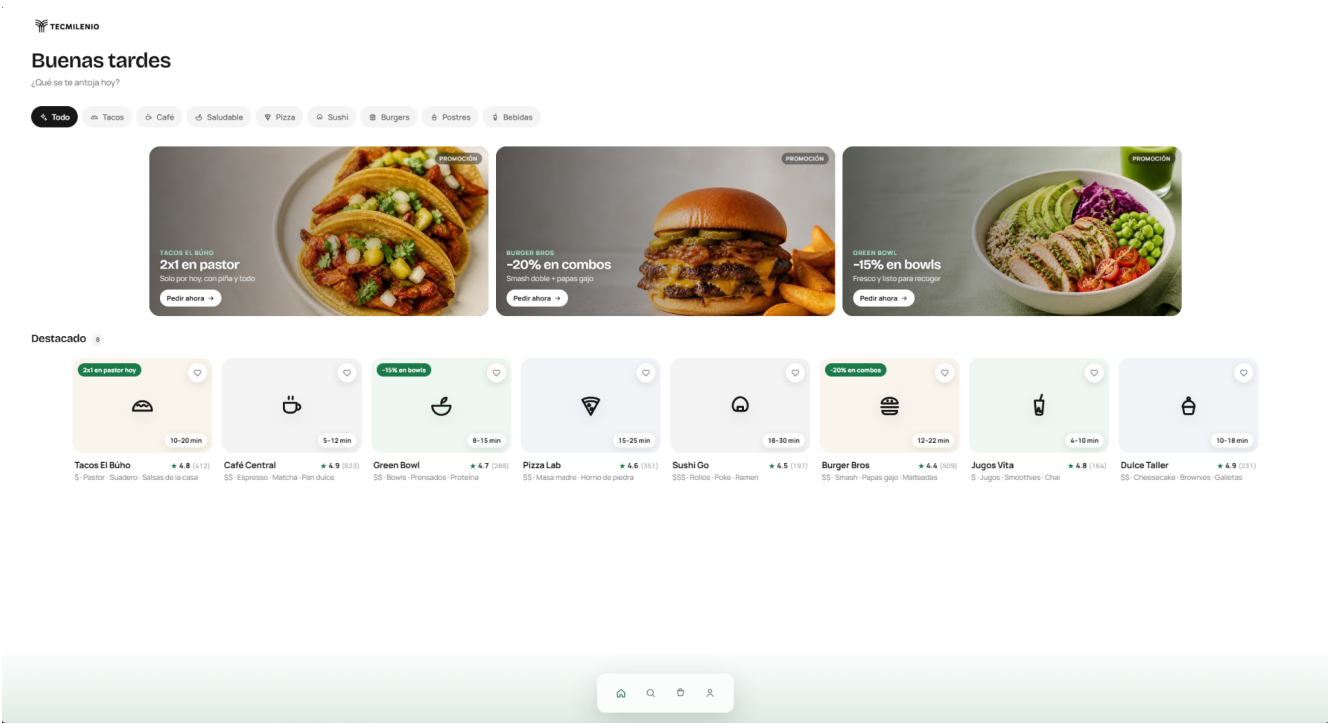


Figura 1. Pantalla de inicio: saludo, categorías, promociones y locales destacados.

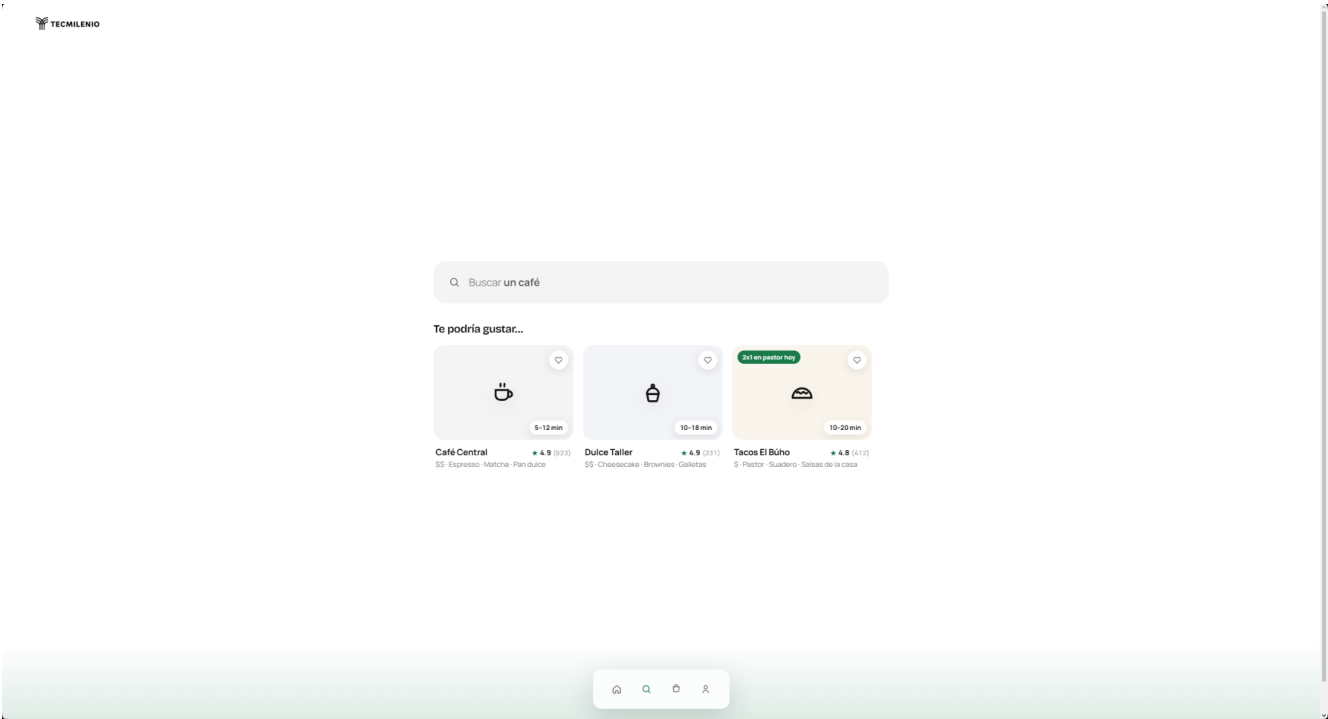


Figura 2. Pantalla de búsqueda: campo de búsqueda y resultados sugeridos.

Elementos observables: navegación inferior, búsqueda, tarjetas de locales, favoritos, promociones, valoración y tiempos estimados de recogida.

8. Casos de uso probados

Los escenarios se basan en el plan de QA del proyecto. Se contemplan autenticación, catálogo, carrito, pago, pedido activo, historial, navegación, accesibilidad y manejo de errores.

ID	Escenario	Pasos	Resultado esperado
CU-01	Inicio y catálogo	Entrar a /client y consultar categorías, promociones y destacados.	Visualizar el inicio correctamente; si la API falla, utilizar el menú local.
CU-02	Búsqueda	Entrar a /client/search, escribir y borrar una consulta.	Filtrar por nombre/tags y abrir la ficha correcta.
CU-03	Configurar pedido	Abrir un platillo, elegir extras/opciones y agregarlo.	Feedback de agregado, navegación al carrito y badge actualizado.
CU-04	Pago con tarjeta	Con sesión API, seleccionar tarjeta y confirmar.	Crear la orden; carrito vacío e island visible; sin conservar datos de tarjeta.
CU-05	Pago en efectivo	Seleccionar efectivo y confirmar.	Crear el pedido y mostrar que se paga al recoger.
CU-06	Pago sin sesión	Intentar pagar sin autenticación real.	Solicitar inicio de sesión y conservar el carrito.
CU-07	Seguimiento	Consultar el island y el estado del pedido.	Reflejar el estado del servidor: recibido → en cocina → listo.
CU-08	Pickup	Abrir el QR/ticket y acudir a la superficie de recogida.	Mostrar ticket con ítems, total y método.
CU-09	Volver a pedir	Desde pedidos recientes, seleccionar Volver a pedir.	Regresar las líneas al carrito.
CU-10	Accesibilidad	Recorrer auth → inicio → ficha → carrito → pago con teclado.	Tab/Enter funcionales, focus visible y navegación accesible.

9. Resultados de las pruebas: esperado vs. observado

El estado registrado en el plan de QA al 29 de agosto de 2026 indica que el frontend apunta a <https://tecmiratti.fly.dev>, pero POST /api/auth/login responde 404 porque ese host sirve la aplicación y no el backend esperado. Por esta razón, los flujos que dependen de autenticación real y pago quedan bloqueados hasta disponer de la API correctamente expuesta.

Prueba	Esperado	Observado	Resultado
Inicio / catálogo	Mostrar catálogo; si la API falla, usar menú local.	El fallback local permite mantener el catálogo cuando la API falla.	OK / condicionado
Login real	Autenticar y entrar a /client.	El endpoint de login en el host indicado devuelve 404.	BLOQUEADO
Forgot password	Enviar instrucciones si existe la cuenta.	La vista contiene el texto, pero no llama a un endpoint porque todavía no existe recuperación real.	BLOQUEADO
Pago	Crear pedido y vaciar carrito sólo después de éxito.	La API necesaria para pago no está disponible en el entorno documentado.	BLOQUEADO

Prueba	Esperado	Observado	Resultado
Fallback de catálogo	Evitar skeleton infinito y mantener menú visible.	El plan registra fallback al menú local.	OK
QR / pickup	Mostrar ticket/QR y validar ID.	El flujo está definido en el cliente; requiere validación integrada con backend activo.	PENDIENTE
Panel /restaurant	Operar cola del restaurante.	No está cubierto por el alcance actual del plan de pruebas.	PENDIENTE

Interpretación: el estado actual demuestra que el cliente web puede presentar y recorrer buena parte de la experiencia con datos locales, pero todavía no permite certificar el flujo transaccional completo. La conexión real de autenticación y pago debe volver a probarse cuando la API esté disponible.

10. Pruebas de calidad y accesibilidad

El plan de QA establece pruebas en desktop (≥ 721 px) y teléfono (≤ 640 px), además de teclado y, cuando sea posible, VoiceOver o NVDA. También contempla contraste WCAG 2.1 AA.

- Teclado: Tab debe recorrer campos, links y botones; Enter debe ejecutar acciones; Escape debe cerrar el QR fullscreen.
- Focus: debe ser visible y no existir una trampa de focus, excepto el overlay QR donde el atrapamiento es intencional.
- Landmarks: usar main, headings en orden y navegación para el dock; el island debe exponer aria-expanded.
- Mensajes: errores con role=alert y anuncios accesibles para pago exitoso o reordenar.
- Contraste: cumplir AA y no depender exclusivamente del color para comunicar estados.
- Touch: targets del dock de al menos 44 px y controles de favoritos/carrito suficientemente separados.

11. Alcance funcional de la API v1

La primera versión de la API concentra el flujo esencial del producto: autenticación, catálogo, búsqueda, favoritos, órdenes, pago y operación del restaurante.

Módulo	Responsabilidad
Auth	Registro, login, JWT y autorización por rol.
Catálogo	Locales, categorías, productos, opciones, extras, precios y disponibilidad.
Búsqueda	Búsqueda y filtros básicos.
Favoritos	Guardar y eliminar productos/locales favoritos.
Órdenes	Crear, consultar y controlar el ciclo de vida de pedidos.
Pago	Tarjeta y efectivo; el sistema propio no debe guardar números de tarjeta.
Restaurante	Cola, estados y confirmación de pickup.
ETA	Estimación calculada por el servidor.

12. Seguridad e integridad de datos

- JWT: proteger endpoints y validar el rol del usuario antes de ejecutar operaciones restringidas.
- Precios: nunca aceptar el total enviado por el cliente como fuente de verdad. Recalcularlo en el servidor.
- Menú: comprobar que cada producto y opción seleccionada existen, pertenecen al local correcto y están disponibles.

- Orden: el servidor genera y controla el identificador, estados, total y relación con el usuario.
- Tarjetas: el cliente no debe guardar números de tarjeta. Los datos sensibles deben manejarse mediante el mecanismo de pago correspondiente.
- Autorización: un restaurante sólo debe poder operar las órdenes y recursos de sus propios locales.
- Transiciones: validar desde servidor los cambios de estado permitidos.
- ETA: la estimación oficial pertenece al servidor y el frontend sólo presenta el resultado.

13. Diagrama del proyecto

El diagrama incluido corresponde a la versión proporcionada para esta documentación.

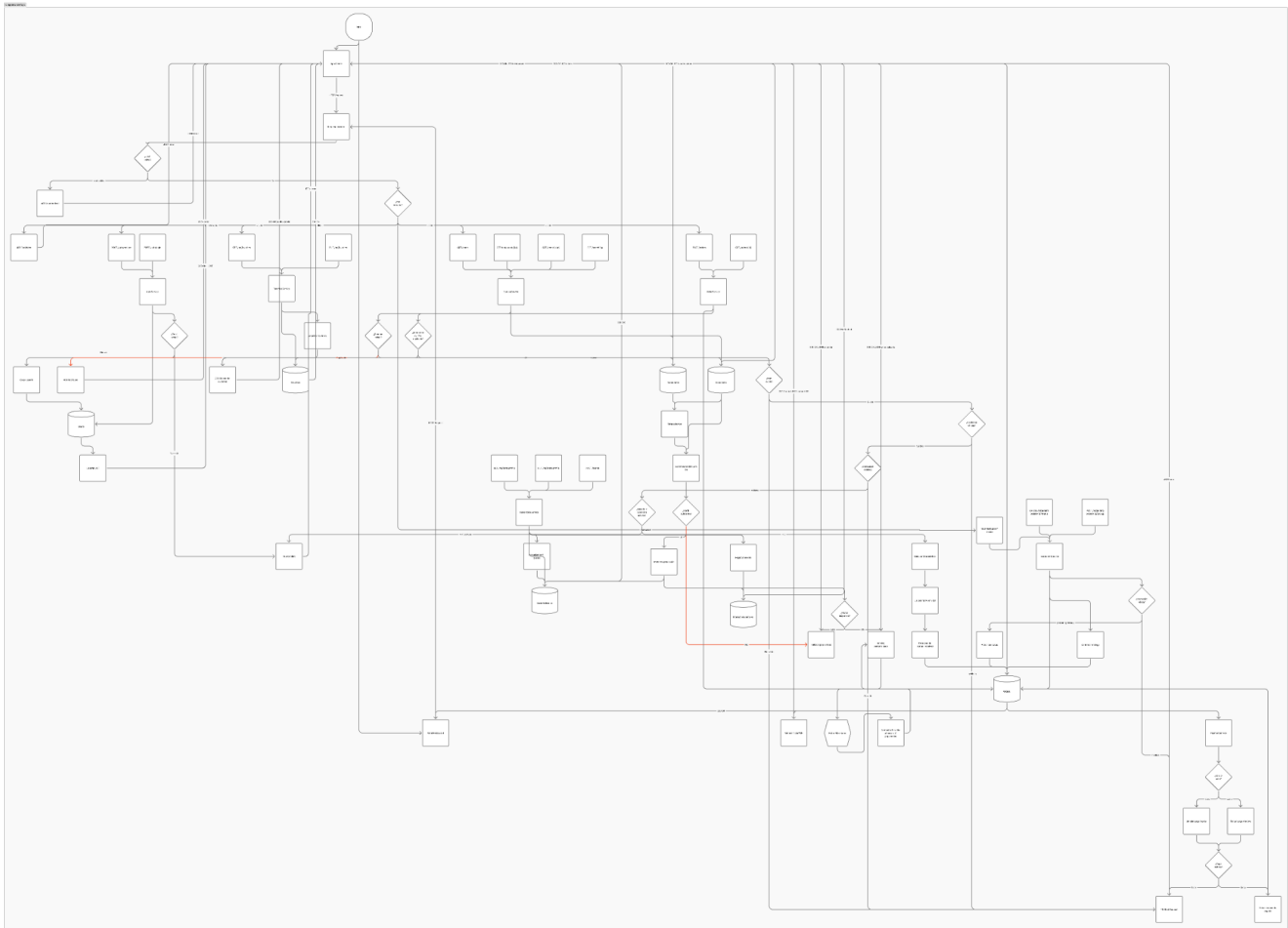


Figura 3. Diagrama proporcionado para TecmiPickup.

El diagrama representa el flujo general del sistema y sus decisiones, procesos, datos y conexiones. Debido a su densidad, se conserva la imagen completa.

14. Resumen del proyecto

Elemento	Definición
Producto	TecmiPickup — pedidos de comida para recoger en campus.
Frontend	TypeScript, Next.js y Tailwind CSS.
Backend	Java, Spring Boot, REST y JWT.
Hosting frontend	Cloudflare.
Servidor / API	Fly.io.
Roles	Cliente y Restaurante.
API v1	Auth, catálogo, búsqueda, favoritos, órdenes, pago, ETA y operación del local.
Modelo	Pedido anticipado, pago, preparación y recogida en mostrador.

Documento actualizado conforme a las modificaciones solicitadas: se eliminaron las secciones de evolución, fuera de alcance, contrato REST y criterios REST; también se actualizó la infraestructura a Cloudflare para el frontend y Fly.io para el servidor.